3. For all stack symbols Y in Γ or $Y = X_0$, $\delta_N(q, \epsilon, Y)$ contains (p, ϵ). By this rule, whenever P_F accepts, P_N can start emptying its stack without consuming any more input.

4. For all stack symbols Y in Γ or $Y = X_0$, $\delta_N(p, \epsilon, Y) = \{(p, \epsilon)\}$. Once in state p, which only occurs when P_F has accepted, P_N pops every symbol on its stack, until the stack is empty. No further input is consumed.

Now, we must prove that w is in $N(P_N)$ if and only if w is in $L(P_F)$. The ideas are similar to the proof for Theorem 6.9. The "if" part is a direct simulation, and the "only-if" part requires that we examine the limited number of things that the constructed PDA P_N can do.

(If) Suppose $(q_0, w, Z_0) \overset{*}{\underset{P_F}{\vdash}} (q, \epsilon, \alpha)$ for some accepting state q and stack string α. Using the fact that every transition of P_F is a move of P_N, and invoking Theorem 6.5 to allow us to keep X_0 below the symbols of Γ on the stack, we know that $(q_0, w, Z_0X_0) \overset{*}{\underset{P_N}{\vdash}} (q, \epsilon, \alpha X_0)$. Then P_N can do the following:

$$(p_0, w, X_0) \underset{P_N}{\vdash} (q_0, w, Z_0X_0) \overset{*}{\underset{P_N}{\vdash}} (q, \epsilon, \alpha X_0) \overset{*}{\underset{P_N}{\vdash}} (p, \epsilon, \epsilon)$$

The first move is by rule (1) of the construction of P_N, while the last sequence of moves is by rules (3) and (4). Thus, w is accepted by P_N, by empty stack.

(Only-if) The only way P_N can empty its stack is by entering state p, since X_0 is sitting at the bottom of stack and X_0 is not a symbol on which P_F has any moves. The only way P_N can enter state p is if the simulated P_F enters an accepting state. The first move of P_N is surely the move given in rule (1). Thus, every accepting computation of P_N looks like

$$(p_0, w, X_0) \underset{P_N}{\vdash} (q_0, w, Z_0X_0) \overset{*}{\underset{P_N}{\vdash}} (q, \epsilon, \alpha X_0) \overset{*}{\underset{P_N}{\vdash}} (p, \epsilon, \epsilon)$$

where q is an accepting state of P_F.

Moreover, between ID's (q_0, w, Z_0X_0) and $(q, \epsilon, \alpha X_0)$, all the moves are moves of P_F. In particular, X_0 was never the top stack symbol prior to reaching ID $(q, \epsilon, \alpha X_0)$.[3] Thus, we conclude that the same computation can occur in P_F, without the X_0 on the stack; that is, $(q_0, w, Z_0) \overset{*}{\underset{P_F}{\vdash}} (q, \epsilon, \alpha)$. Now we see that P_F accepts w by final state, so w is in $L(P_F)$. □

6.2.5 Exercises for Section 6.2

Exercise 6.2.1 : Design a PDA to accept each of the following languages. You may accept either by final state or by empty stack, whichever is more convenient:

* a) $\{0^n1^m0^n | m, n \geq 1\}$

b) $\{0^n1^m0^m1^n | m, n \geq 1\}$

[3] Although α could be ϵ, in which case P_F has emptied its stack at the same time it accepts.

c) The set of all strings of 0's and 1's so that the number of 1's is twice the number of 0's.

! **Exercise 6.2.2:** Design a PDA to accept each of the following languages.

* a) $\{a^i b^j c^k \mid i = j \text{ or } j = k\}$. Note that this language is different from that of Exercise 5.1.1(b).

b) The set of all strings with twice as many 0's as 1's.

!! **Exercise 6.2.3:** Design a PDA to accept each of the following languages.

a) $\{a^i b^j c^k \mid i \neq j \text{ or } j \neq k\}$.

b) The set of all strings of a's and b's that are *not* of the form ww, that is, not equal to any string repeated.

*! **Exercise 6.2.4:** Let P be a PDA with empty-stack language $L = N(P)$, and suppose that ϵ is not in L. Describe how you would modify P so that it accepts $L \cup \{\epsilon\}$ by empty stack.

Exercise 6.2.5: PDA $P = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \{X, Y, Z\}, \delta, q_0, Z.\{q_3\})$ has the following rules defining δ:

$$\begin{array}{ll}
\delta(q_0, \epsilon, Z) = \{(q_1, XZ)\} & \delta(q_2, 0, Y) = \{(q_2, \epsilon)\} \\
\delta(q_1, 0, X) = \{(q_1, YX)\} & \delta(q_2, \epsilon, X) = \{(q_2, \epsilon)\} \\
\delta(q_1, 0, Y) = \{(q_1, YY)\} & \delta(q_1, \epsilon, Z) = \{(q_3, Z)\} \\
\delta(q_1, 1, Y) = \{(q_2, Y)\} & \delta(q_2, \epsilon, Z) = \{(q_3, Z)\} \\
\delta(q_2, 1, Y) = \{(q_2, Y)\} &
\end{array}$$

* a) Give an execution trace (sequence of ID's) showing that string 0110 is in $L(P)$.

b) Gives an execution trace showing that 00100 is in $L(P)$.

c) Give the contents of the stack after P has read $0^3 1^5 0^3$ from its input.

! d) Informally describe $L(P)$.

Exercise 6.2.6: Consider the PDA P from Exercise 6.1.1.

a) Convert P to another PDA P_1 that accepts by empty stack the same languages that P accepts by final state; i.e. $N(P_1) = L(P)$.

b) Find a PDA P_2 such that $L(P_2) = N(P)$; i.e. P accepts by final state what P accepts by empty stack.

! **Exercise 6.2.7:** Show that if P is a PDA, then there is a PDA P_2 with only two stack symbols, such that $L(P_2) = L(P)$. *Hint*: Binary-code the stack alphabet of P.

*! **Exercise 6.2.8:** A PDA is called *restricted* if on any transition it can increase the height of the stack by at most one symbol. That is, for any rule $\delta(q, a, Z)$ contains (p, γ), it must be that $|\gamma| \leq 2$. Show that if P is a PDA, then there is a restricted PDA P_3 such that $L(P) = L(P_3)$.

Exercise 6.3.4: Convert the PDA of Exercise 6.1.1 to a context-free grammar.

Exercise 6.3.5: Below are some context-free languages. For each, devise a PDA that accepts the language by empty stack. You may, if you wish, first construct a grammar for the language, and then convert to a PDA.

a) $\{a^n b^m c^{2(n+m)} \mid n \geq 0,\ m \geq 0\}$.

b) $\{a^i b^j c^k \mid i = 2j \text{ or } j = 2k\}$.

! c) $\{0^n 1^m \mid n \leq m \leq 2n\}$.

*! **Exercise 6.3.6:** Show that if P is a PDA, then there is a one-state PDA P_1 such that $N(P_1) = N(P)$.

! **Exercise 6.3.7:** Suppose we have a PDA with s states, t stack symbols, and no rule in which a replacement stack string has length greater than u. Give a tight upper bound on the number of variables in the CFG that we construct for this PDA by the method of Section 6.3.2.

6.4 Deterministic Pushdown Automata

While PDA's are by definition allowed to be nondeterministic, the deterministic subcase is quite important. In particular, parsers generally behave like deterministic PDA's, so the class of languages that can be accepted by these automata is interesting for the insights it gives us into what constructs are suitable for use in programming languages. In this section, we shall define deterministic PDA's and investigate some of the things they can and cannot do.

6.4.1 Definition of a Deterministic PDA

Intuitively, a PDA is deterministic if there is never a choice of move in any situation. These choices are of two kinds. If $\delta(q, a, X)$ contains more than one pair, then surely the PDA is nondeterministic because we can choose among these pairs when deciding on the next move. However, even if $\delta(q, a, X)$ is always a singleton, we could still have a choice between using a real input symbol, or making a move on ϵ. Thus, we define a PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ to be *deterministic* (a deterministic PDA or DPDA), if and only if the following conditions are met:

1. $\delta(q, a, X)$ has at most one member f... ...d X in Γ.

use. In practice, another mechanism, such as a "symbol table" is used to record declared identifiers, and we do not try to design a parser that, by itself, checks for "definition prior to use."

Example 7.21: Let $L = \{ww \mid w \text{ is in } \{0,1\}^*\}$. That is, L consists of repeating strings, such as ϵ, 0101, 00100010, or 110110. If L is context-free, then let n be its pumping-lemma constant. Consider the string $z = 0^n1^n0^n1^n$. This string is 0^n1^n repeated, so z is in L.

Following the pattern of the previous examples, we can break $z = uvwxy$, such that $|vwx| \leq n$ and $vx \neq \epsilon$. We shall show that uwy is not in L, and thus show L not to be a context-free language, by contradiction.

First, observe that, since $|vwx| \leq n$, $|uwy| \geq 3n$. Thus, if uwy is some repeating string, say tt, then t is of length at least $3n/2$. There are several cases to consider, depending where vwx is within z.

1. Suppose vwx is within the first n 0's. In particular, let vx consist of k 0's, where $k > 0$. Then uwy begins with $0^{n-k}1^n$. Since $|uwy| = 4n - k$, we know that if $uwy = tt$, then $|t| = 2n - k/2$. Thus, t does not end until after the first block of 1's; i.e., t ends in 0. But uwy ends in 1, and so it cannot equal tt.

2. Suppose vwx straddles the first block of 0's and the first block of 1's. It may be that vx consists only of 0's, if $x = \epsilon$. Then, the argument that uwy is not of the form tt is the same as case (1). If vx has at least one 1, then we note that t, which is of length at least $3n/2$, must end in 1^n, because uwy ends in 1^n. However, there is no block of n 1's except the final block, so t cannot repeat in uwy.

3. If vwx is contained in the first block of 1's, then the argument that uwy is not in L is like the second part of case (2).

4. Suppose vwx straddles the first block of 1's and the second block of 0's. If vx actually has no 0's, then the argument is the same as if vwx were contained in the first block of 1's. If vx has at least one 0, then uwy starts with a block of n 0's, and so does t if $uvw = tt$. However, there is no other block of n 0's in uwy for the second copy of t. We conclude in this case too, that uwy is not in L.

5. In the other cases, where vwy is in the second half of z, the argument is symmetric to the cases where vwx is contained in the first half of z.

Thus, in no case is uwy in L, and we conclude that L is not context-free. □

7.2.4 Exercises for Section 7.2

Exercise 7.2.1: Use the CFL pumping lemma to show each of these languages not to be context free:

* a) $\{0^n \mid n \text{ is a prime}\}$.

b) $\{0^n1^n0^n \mid n \geq 1\}$.

c) $\{0^m1^n \mid m \neq n\}$. *Hint*: Adapt the same ideas used in Example 4.3, which showed this language not to be regular.

*! d) $\{0^i1^j \mid j = i^2\}$.

! e) $\{a^nb^nc^i \mid n \leq i \leq 2n\}$.

! f) $\{ww^Rw \mid w \text{ is a string of 0's and 1's}\}$. That is, the set of strings consisting of some string w followed by the same string in reverse, and then the string w again, such as 001100001.

Exercise 7.2.2: When we try to apply the pumping lemma to a CFL, the "adversary wins," and we cannot complete the proof. Show what goes wrong when we choose L to be one of the following languages:

a) $\{010, 11\}$

* b) The set of all strings with equal number of 0 's and 1's

* c) The set of all strings of balanced parentheses.

! **Exercise 7.2.3:** There is a stronger version of the CFL pumping lemma known as *Ogden's lemma*. It differs from the pumping lemma we proved by allowing us to focus on any n "distinguished" positions of a string z and guaranteeing that the strings to be pumped have between 1 and n distinguished positions. The advantage of this ability is that a language may have strings consisting of two parts, one of which can be pumped without producing strings not in the language, while the other *does* produce strings outside the language when pumped. Without being able to insist that the pumping take place in the latter part, we cannot complete a proof of non-context-freeness. The formal statement of Ogden's lemma is: If L is a CFL, then there is a constant n, such that if z is any string of length at least n in L, in which we select at least n positions to be *distinguished*, then we can write $z = uvwxy$, such that:

1. vwx has at most n distinguished positions.

2. vx has at least one distinguished position.

3. For all i, uv^iwx^iy is in L.

Prove Ogden's lemma. *Hint*: The proof is really the same as that of the pumping lemma of Theorem 7.18 if we pretend that the nondistinguished positions of z are not present as we select a long path in the parse tree for z.

* **Exercise 7.2.4:** Use Ogden's lemma (Exercise 7.2.3 to simplify the proof in Example 7.21 that $L = \{ww \mid w \text{ is in } \{0,1\}^*\}$ is not a CFL. *Hint*: With $z = 0^n1^n0^n1^n$, make the two middle blocks distinguished.